

Integration and Testing RPS APIs on QA

- [How to integrate in raptor application and do local testing?](#)

• **Profile Attributes**

◦ Steps to Test

- i. Check if the metadata for the attributes your using are available on staging. Following are the ways to verify the same.

1. Using the Portal:

- a. <https://www.msmaster.qa.paypal.com/campaigninsights/segments/attributes>. Search for the attribute you want to use and make sure the same is available.
- b. If the attribute is not available, please reach out to DL-PP-rps-platform-dev for help.

- ii. **Update Attributes API:**

1. If you intend to update attributes into RPS using the API, following are the API details for the same:

- a. **Update Attributes**

```
1 METHOD: POST
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile-attributes
3 HEADERS: Content-Type : application/json
4 REQUEST:
5 {
6   "cust_id": "12047829602241376831", //Account number of the user
7   "profile_attributes": [
8     {
9       "long_name": "account_creation_channel", //name of the attribute.
10      "value": "TESLA"
11    },
12    {
13      "long_name": "acct_cre_dt",
14      "value": "2016-12-10"
15    }
16  ]
17 }
18 RESPONSE: [STATUS CODE: 200 for successful Update and 400 for a validation error]
19
```

NOTE: If you need help with getting the name of the attribute you should pass in the above request, please reach out to our team. The same also maps to the "System Name" in the below section on Segmentation Portal. Double click and copy the system name from the portal

✖ **Oops, Diagram Unavailable**

This diagram can't be displayed. It may have been moved or deleted, or the page's access settings may be preventing it from loading. Pages with restricted access must also include Gliffy Diagrams in their allowed list.

- iii. **Retrieve the attributes associated with an identity.**

1. If you want to retrieve an attribute on live from our system, using the following end point:

- a. **Get Attributes**

```
1 METHOD: GET
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile-attributes?
3 id_type=ACCOUNT_NUMBER&id_value=12047829602241376831&filter_attr=cause_recommendation
4 [NOTE: pass the name of the attribute against the query param to retrieve the same]
5 HEADERS: Content-Type : application/json
```

```

6 RESPONSE: [STATUS CODE: 200 / 404]
7 {
8   "cust_id": "12047829602241376831",
9   "profile_attributes": [
10    {
11      "short_name": "cr",
12      "long_name": "cause_recommendation",
13      "value": [
14        "C1",
15        "C2"
16      ],
17      "timestamp": 1507336654077
18    }
19  ],
20  "pid": "b9d3b51b-6ace-4886-ae03-8416311f6d3d"
21 }
22

```

The request for encrypted account number works. You can now use above command by using `id_type=ENCRYPTED_ACCOUNT_NUMBER` and `id_value` as the encrypted value.

iv. Postman collection with Sample Calls.

1. [🔗 Profile Attribute Online RPS APIs v2](#) [🔗 Profil_Attribute_Online_RPS_APIs_v2](#)

v. Some additional details: [Customer Profile Attributes](#)

vi. Link to check metadata of existing attributes on PRODUCTION:

<https://internal.paypalinc.com/campaigninsights/attributes>

vii. Update attributes of LIST/MAP data type using kafka.

1. We currently do not support direct API updates for MAP/LIST based attributes, hence the following steps can be followed for testing updation of such attributes on staging:
2. Install kafka locally. Follow the steps mentioned here: <https://kafka.apache.org/quickstart>
3. Once installed go into the folder "kafka_2.11-0.10.1.0"
 - a. command to publish a message;

```

1 bin/kafka-console-producer.sh --broker-list gc-mskafka1002.pp-devqa-ms-thirdparty.us-
  central1.gcp.dev.paypalinc.com:9092 --topic consumer.rps.profile-slca
2
3 //the broker (Eg: <gc-mskafka1002.pp-devqa-ms-thirdparty.us-central1.gcp.dev.paypalinc.com>) to be
  passed in the above command can be picked from : https://10.176.146.162:8443/kafkamonitor/cluster?
  selectedCluster=GCPIntegrationCluster. Preferable to pick the master broker

```

b. Sample message to publish:

```

1 //Sample message:
2 {"type": "customerid", "id": "functest", "values":
3  [{"type": "AttributeUpdateV2", "timestamp": 1498774621207, "values":
4    [{"cust_id": "168918769392", "profile_attributes":
5      [{"long_name": "cntr_merc_paypal_shoppers_percentage", "value": {"1e0b3556-ff9c-47c1-9f74-
6        19f7356d0d4a": 60, "1da105ad-3e21-4dde-ba58-554b3d708849": 75, "29923693-6647-4348-a58b-71bbb9f18f6
7        ": 40}}], "pid": null}}]}
8
9 Note:
10 You can replace the following in the above message to test updating your attribute:
11 1. "cust_id": "168918769392" --> replace account number
12 2. "long_name" --> use the long_name relevant to your application. Also, provide the appropriate
13    value for the attribute against the "value" field.

```

NOTE :publish the same message 2-3 times so the publishing goes through successfully. Once, published you can use the GET API to read the data and verify.

viii. Update attributes via kafka

- [Attribute Update using Kafka](#)

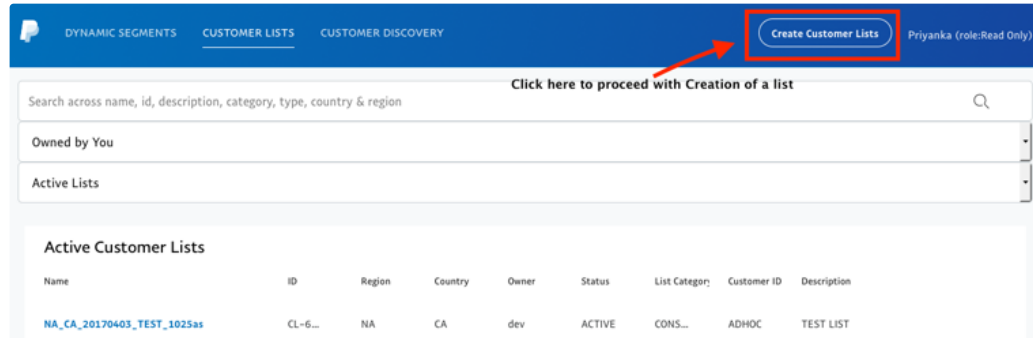
• Customer Lists

- Steps to Test :

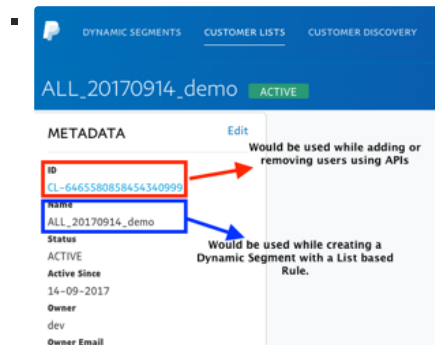
i. Create a Customer List

1. Using the Portal:

- Link : <https://www.msmaster.qa.paypal.com/campaigninsights/lists/view>
- Screen Shot:



- Once a List is Created the customer list id and code are as shown below. These would be used for subsequent segment creation or add/remove users from a list:



2. Using the Customer List Definition API directly. (Using the tool would be easier)

Create Customer List Definition API

```
1 METHOD: POST
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/customer-lists
3 HEADERS: Content-Type : application/json
4
5 REQUEST:
6 {
7   "code": "NA_CA_20170403_TEST_DEMO",
8   "description": "TEST LIST",
9   "countries": [
10    "CA"
11  ],
12  "regions": [
13    "NA"
14  ],
15  "type": "ADHOC",
16  "category": "CONSUMER",
17  "created_by": "dev",
18  "expiry_ts": "1495477378304",
19  "lifecycle_status": "LIVE"
20 }
21
22 RESPONSE: [STATUS CODE: 200]
23 {
24   "id": "CL-6468413433652722168",
25   "code": "NA_CA_20170403_TEST_DEMO",
26   "description": "TEST LIST",
27   "type": "ADHOC",
```

```

28  "category": "CONSUMER",
29  "create_ts": "1506044863355",
30  "update_ts": "1506044863355",
31  "member_count": 0,
32  "countries": [
33    "CA"
34  ],
35  "regions": [
36    "NA"
37  ],
38  "created_by": "dev",
39  "expiry_ts": "1495477378304",
40  "lifecycle_status": "LIVE"
41  }
42

```

NOTE: Currently we have a limitation that the lists created in production using the tool might not have the same name and id in the QA environment. Hence, you would have to recreate similar lists on staging and test using those.

ii. Add/Remove members from a Customer list

1. Using APIs mentioned below (NOTE: On staging we do not support bulk load of users and hence you can simulate adding or removing users from a list using the APIs).
2. API to add a user to a list.

Add a user to a Customer List

```

1  METHOD: POST
2  URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/customer-lists/CL-
6465007365187988655/members (Note: use the customer list for your list in the url)
3  HEADERS: Content-Type : application/json
4
5  REQUEST:
6  {
7      "id_type": "ACCOUNT_NUMBER",
8      "id_value": "1608754337057906945" //account number of the user to be added
9  }
10
11 RESPONSE: [STATUS CODE: 200]
12

```

Note: If you receive an error stating "Customer Id Not Found", then please add the user into RPS by setting an attribute for the user. Use APIs defined here to update any attribute for the user:

[Integration and Testing RPS APIs on QA#UpdateAttributesAPI:](#)

3. API to remove a user from a list.

Remove a user from a customer List

```

1  METHOD: DELETE
2  URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/customer-lists/CL-
6465007365187988655/members (Note: use the customer list for your list in the url)
3  HEADERS: Content-Type : application/json
4
5  REQUEST:
6  {
7      "id_type": "ACCOUNT_NUMBER",
8      "id_value": "1608754337057906945"
9  }
10
11 RESPONSE: [STATUS CODE: 204]
12

```

iii. Postman collection with Sample Calls.

1. For Customer List Definition Related APIs
 - [CustomerList_new.postman_collection.json](#)
2. For Customer List Membership Related APIs

- `CustomerListMembership.postman_collection.json`

iv. Some additional details: [Customer List](#)

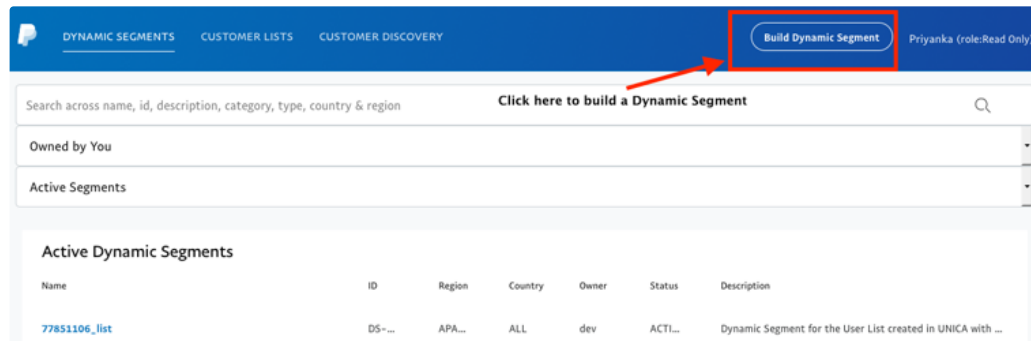
• Dynamic Segments

- Steps to Test :

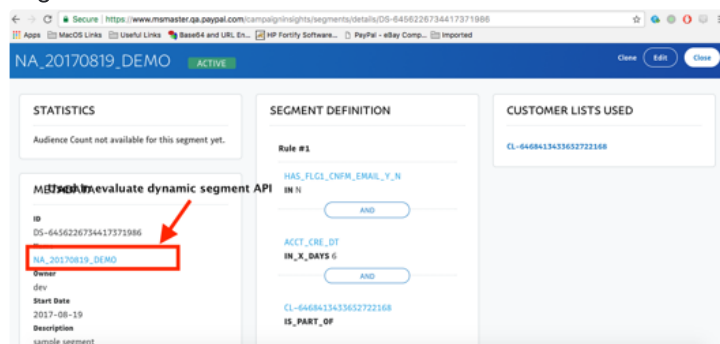
i. Create a Dynamic Segment

1. Using the Portal:

- Link : <https://www.msmaster.qa.paypal.com/campaigninsights/lists/view>
- Screen Shot:



- Once a Segment is Created the segment code is as shown below. This would be used for subsequent segment evaluation:



- Video around segment creation can be found here :
 - https://paypal-my.sharepoint.com/:f/p/sbhagatwala/EnpQjo9c7XdBpv5F7_g2I4BrCNnxRo8HSo5N_Fb-Easzwe=5%3a88pM6f&at=9

NOTE: Currently we have a limitation that the segments created in production using the tool might not have the same name and id in the QA environment. Hence, you would have to recreate similar segments on staging and test using those.

ii. API to evaluate if a user belong to a dynamic segment:

1. Generic API to evaluate both dynamic segments and customer lists.

This is a generic evaluate end point, where you can pass either customer list codes or dynamic segment codes and

evaluate if a given user is a part of the same. This API gives you the capability to use the same API to evaluate against

segments or lists, without having to differentiate the same.

Generic Evaluate End Point

```
1 METHOD: POST
2 HEADERS: Content-Type : application/json
3 URL: https://msmaster.qa.paypal.com:20068/v1/dynsegmentationserv/evaluate
4
5 REQUEST:
6 {
7   "id_type": "ACCOUNT_NUMBER",
8   "id": "1617082364825547238",
9   "segment_codes": [
10    "acct_creation_channel_hermes",
11    "NA_20170922_Ravi_Test_DS",
12    "NA_20170922_Ravi_Test_CL",
13    "DUMMY"
14   ]
15 }
16
17 NOTE: For encrypted account number based evaluation use id_type as ENCRYPTED_ACCOUNT_NUMBER
18       For cookie id or fpti id based evaluation use id_type as COOKIE_ID
19
20 RESPONSE: [STATUS CODE: 200]
21 {
22   "id_type": "ACCOUNT_NUMBER",
23   "id": "1617082364825547238",
24   "segment_codes": [
25     "NA_20170922_Ravi_Test_DS",
26     "NA_20170922_Ravi_Test_CL"
27   ]
28 }
```

iii. Postman collection with Sample Calls.

1. [dynsegments_evaluate_postman_collection](#) .json

iv. Some additional details: [Dynamic Segments](#)

• Identity Mappings

◦ Retrieve user id from multiple identities.

- The API is used to identify the user account number linked to a given set of identities.
- Currently the API accepts COOKIE_ID and IP.
- More details around the APIs and testing are documented here: [Identity Mapping Enhancements in RPS#TestingonStaging](#)

◦ Update Id mappings on staging : **[DEPRECATED]**

- Legacy ID Mapping APIs in RPS: We currently support association of cookie/fpti ids with account numbers in RPS. In production, we have pipelines set that update these id mappings constantly. These data updates only happen in the LIVE environment. However, on staging if you intend to associate an identity and retrieve details of a user based on other identities , following are the APIs you can use.

Updaye Identity Mappings on Staging

```
1 METHOD: POST
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile?clientid=rps
3 HEADERS: Content-Type : application/json
4
5 REQUEST:
6 {
7   "keys": [
8     {
9       "name" : "pu",
10      "value" : "2235611535848528047"
11     },
12     {
13       "name" : "fpti",
14       "value" : "djsqfjkdysuif6we86ngjgdss"
15     }
16   ]
17 }
```

```

17 ]
18 }
19 }
20
21 RESPONSE: [STATUS CODE: 200/201]
22

```

- Retrieve the account number given a cookie id **[DEPRECATED]**

▪ Retrieve User linked to an Identity

```

1 METHOD: GET
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile/identity?
   id_type=COOKIE_ID&id_value=jdsykylsidiyiiteisytilysifktgdsu
3 HEADERS: Content-Type : application/json
4
5 RESPONSE: [STATUS CODE: 200/404]
6 {
7   "account_number": "2235611535848528047"
8 }

```

▪ Sample Postman Collection:

- [id_mapCollection](#)

◦ Update identities using kafka.

- To publish id mappings with score:
- Install kafka locally. Follow the steps mentioned here: <https://kafka.apache.org/quickstart>
- Once installed go into the folder "kafka_2.11-0.10.1.0"
 - command to publish a message;

```

1 bin/kafka-console-producer.sh --broker-list gc-mskafka1004.pp-devqa-ms-thirdparty.us-
   central1.gcp.dev.paypalinc.com:9092,gc-mskafka1007.pp-devcos-kafka-qa.us-
   central1.gcp.dev.paypalinc.com:9092,gc-mskafka1008.pp-devcos-kafka-qa.us-
   central1.gcp.dev.paypalinc.com:9092,gc-mskafka1009.pp-devcos-kafka-qa.us-
   central1.gcp.dev.paypalinc.com:9092,gc-mskafka1006.pp-devcos-kafka-qa.us-
   central1.gcp.dev.paypalinc.com:9092, gc-mskafka1005.pp-devcos-kafka-qa.us-
   central1.gcp.dev.paypalinc.com:9092 --topic consumer.rps.profile
2
3 //the broker (Eg: <gc-mskafka1002.pp-devqa-ms-thirdparty.us-central1.gcp.dev.paypalinc.com>) to be
   passed in the above command can be picked from : https://10.176.146.162:8443/kafkamon/cluster?
   selectedCluster=GCPIntegrationCluster. Preferable to pick the master broker

```

2. Sample message to publish:

```

1 //Sample message:
2 For Venmo id - ip with score mapping:
3 {"type":"pu","id":"636318","values":[{"type":"idMapUpdate","timestamp":1511735263987,"values":
   {"keys":[{"name":"ip","value":"10.0.0.1","score":"10"},
   {"name":"vpu","value":"123456","score":null}],"segments":null}}}
4
5 Note:
6 You can replace the following in the above message to test updating your identity:
7 1. "name":"ip","value":"10.0.0.1","score":"10" --> provide ip,value and score that you plan to use.
8 2. "name":"vpu","value":"123456" --> provide value of venmo id you wish to link the ip with.
9
10
11 //Sample message:
12 For paypal id - ip with score mapping:
13 {"type":"pu","id":"test","values":[{"type":"rpsIpMap","timestamp":1511735263987,"values":{"keys":
   [{"name":"ip","value":"1.0.0.0","score":"60.0"},
   {"name":"pu","value":"26541782572354785","score":null}],"segments":null}}}
14 Note:
15 You can replace the following in the above message to test updating your identity:
16 1. "name":"ip","value":"1.0.0.0","score":"60.0" --> provide ip,value and score that you plan to use.
17 2. "name":"pu","value":"26541782572354785" --> provide value of paypal id you wish to link the ip
   with.
18
19 //Sample Message for cookie id mappings:
20 {"type":"guid","id":"feedb24c1760a4a17f71c65387ffffffff","values":
   [{"type":"rpsFptiRequest","timestamp":1610757111913,"values":{"keys":
   [{"name":"pu","value":"12841783706582432","score":null},
   {"name":"fpti","value":"feedb24c1760a4a17f71c4a7ffffffff","score":null}],"segments":null}}}

```

```

21 You can replace the following in the above message to test updating your identity:
22 1. "name":"fpti","value":"feedb24c1760a4a17f71c4a7ffffff" --> provide cookie id value that you plan
    to use.
23 2. "name":"pu","value":"12841783706582432" --> provide value of paypal id you wish to link the cookie
    id.
24
25

```

NOTE :publish the same message 2-3 times so the publishing goes through successfully. Once, published you can use the GET API to read the data and verify.

• **Legacy Segment Updates *[DEPRECATED]***

- If you intend to update and test legacy rps segments, refer to the following confluence link:
- Sample Calls:
 - GET SEGMENTS:
 - Get Legacy segments using segment id:

◦ **Get segments by id**

```

1 METHOD: GET
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile?
    key=pu:2235611535848528047&clientid=rps&filtersegments=pp.10,pp.11
3 HEADERS: Content-Type : application/json
4
5 RESPONSE: [STATUS CODE: 200/204]
6 {
7   "segments": [
8     {
9       "name": "pp.10",
10      "value": "1"
11    },
12    {
13      "name": "pp.11",
14      "value": "1"
15    }
16  ],
17  "keys": {
18    "pu": "2235611535848528047"
19  },
20  "profileId": "3c30285a-823f-4675-809c-4293eceb78b4"
21 }

```

- Get Legacy segments using segment code:
 - **Get segments by name**

```

1 METHOD: GET
2 URL: https://msmaster.qa.paypal.com:14751/rpsreadserv/v1/profile?
    key=pu:2235611535848528047&clientid=rps&usesegmentname=true&filtersegments=SIGNUP_RECENCY,MARKETING
    _CLASSIFICATION
3 HEADERS: Content-Type : application/json
4
5 RESPONSE: [STATUS CODE: 200/204]
6 {
7   "segments": [
8     {
9       "name": "SIGNUP_RECENCY",
10      "value": "1"
11    },
12    {
13      "name": "MARKETING_CLASSIFICATION",
14      "value": "1"
15    }
16  ],
17  "keys": {
18    "pu": "2235611535848528047"
19  },
20  "profileId": "3c30285a-823f-4675-809c-4293eceb78b4"
21 }

```

- Refer Here for steps to test these APIs:
 - [Testing RPS on msmaster](#)

- **NOTE: We strongly encourage clients to integrate and start using the dynamic segments and related APIs mentioned on this page and stop using the legacy segments. We would not be making any further enhancements or additions to the legacy platform and all further changes or requirements would only be made on the new dynamic segmentation platform. Hence, kindly refrain from making any new integration with the legacy segment APIs, unless there is a real need for the same and kindly request you to review the same with the team.**

- **Segment Filters**

- The API is used to retrieve dynamic segments or customer list based on the filter condition.
 - Sample calls.
 - SEARCH SEGMENTS - Filter based on Customer List

```
1  METHOD: POST
2  URL: https://msmaster.qa.paypal.com:20068/v1/dynsegmentationserv/segments/search
3  HEADERS: Content-Type : application/json
4  REQUEST:
5  {"filters": {
6      "type": ["customer_list"],
7      "codes": ["QA"],
8      "region": ["NA"],
9      "country": ["US"]
10     }, "fields": ["id", "code"] }
11
12  RESPONSE:
13  {
14      "dynamic_segments_count": 0,
15      "customer_lists_count": 3,
16      "customer_lists": [
17          {
18              "id": "CL-6727667954922777823",
19              "code": "NA_20190821_LEAP_QA"
20          },
21          {
22              "id": "CL-7599745265529944730",
23              "code": "PIE_NA_US_REGION_RQA"
24          },
25          {
26              "id": "CL-7062908965644907687",
27              "code": "QA_InStore_Offers_EmployeePilot_AllowList_CL"
28          }
29      ]
30  }
31
32
```

- SEARCH SEGMENTS - Filter based on Dynamic Segments

```
1  METHOD: POST
2  URL: https://msmaster.qa.paypal.com:20068/v1/dynsegmentationserv/segments/search
3  HEADERS: Content-Type : application/json
4  REQUEST:
5  { "filters": {
6      "type": ["dynamic_segment"],
7      "codes": ["QA_"],
8      "region": ["NA"],
9      "country": ["US"]
10     } }
11
12  RESPONSE:
13  {
14      "dynamic_segments_count": 2,
15      "customer_lists_count": 0,
16      "dynamic_segments": [
17          {
18              "id": "DS-7548241186324950614",
19              "code": "QA_560557_Debit_X0_Offer",
20              "audience_count": -1,
21              "type": "BATCH"
22          },
23          {
24
25
```

```

24         "id": "DS-7062909544007308656",
25         "code": "QA_InStore_Offers_EmployeePilot_AllowList_DS",
26         "audience_count": 2,
27         "type": "BATCH"
28     }
29 ]
30 }
31
32

```

- SEARCH SEGMENTS - Optional filter based on dynamic segment and customer list.

```

1  METHOD: POST
2  URL: https://msmaster.qa.paypal.com:20068/v1/dynsegmentationserv/segments/search
3  HEADERS: Content-Type : application/json
4  REQUEST:
5  {
6      "filters": {
7          "type": [],
8          "created_by": ["chpereira"]
9      },
10     "fields": ["id", "code"],
11     "sort_order": "asc",
12     "sort_by": "code"
13 }
14
15 RESPONSE:
16 {
17     "dynamic_segments_count": 1,
18     "customer_lists_count": 2,
19     "dynamic_segments": [
20         {
21             "id": "DS-7062909544007308656",
22             "code": "QA_InStore_Offers_EmployeePilot_AllowList_DS"
23         }
24     ],
25     "customer_lists": [
26         {
27             "id": "CL-7062908965644907687",
28             "code": "QA_InStore_Offers_EmployeePilot_AllowList_CL"
29         },
30         {
31             "id": "CL-7598561682218172651",
32             "code": "chpereira-visa-visa-plus-test"
33         }
34     ]
35 }
36
37

```

- SEARCH SEGMENTS -Additional fields in response for dynamic segment and customer list.

```

1  METHOD: POST
2  URL: https://msmaster.qa.paypal.com:20068/v1/dynsegmentationserv/segments/search
3  HEADERS: Content-Type : application/json
4  REQUEST:
5  {
6      "filters": {
7          "type": []
8      },
9      "fields": ["id", "code", "description", "audience_count", "created_by", "type"],
10 }
11 RESPONSE:
12 {
13     "dynamic_segments_count": 5670,
14     "customer_lists_count": 400,
15     "dynamic_segments": [
16         {
17             "id": "DS-7306801429721457076",
18             "code": "0.5Back_Debit_Card_Eligibility",
19             "description": null,
20             "audience_count": -1,
21             "created_by": "rgovindaraju",
22             "type": "BATCH"

```

```

23     },
24     {
25         "id": "DS-7574376819299418380",
26         "code": "1099DA_2025_Biz",
27         "description": "Segment for merchants with 1099DA applicable",
28         "audience_count": -1,
29         "created_by": "kkhoo",
30         "type": "BATCH"
31     },
32     {
33         "id": "DS-7573582297132770337",
34         "code": "1099DA_2025_Con",
35         "description": "Consumers with Form 1099DA applicable for 2025",
36         "audience_count": -1,
37         "created_by": "kkhoo",
38         "type": "BATCH"
39     },
40     .
41     .
42     .
43     {
44         "id": "CL-6466114797668341538",
45         "code": "27428822_list",
46         "description": "TEST LIST",
47         "audience_count": 0,
48         "created_by": "tnatarajan",
49         "type": "ADHOC"
50     },
51     .
52     .
53
54

```

How to integrate in raptor application and do local testing?

application.properties (Covers Sandbox and Prod)

```

rpsreadserv.baseUrl=https://${rpsreadserv_host}:${rpsreadserv_port}
rpsreadserv.profile_attributes=/rpsreadserv/v1/profile-attributes
rpsreadserv.attributes_metadata=/rpsreadserv/v1/metadata
rpsreadserv.http.connectionTimeoutInMs=${default_connection_timeout_ms}
rpsreadserv.http.readTimeoutInMs=${default_timeout_ms}
rpsreadserv.http.requestTimeoutInMs=${default_timeout_ms}
rpsreadserv.http.sslHandshakeTimeoutInMs=${default_sslHandshake_timeout_ms}
rpsreadserv.http.connectionRequestTimeoutInMs=${default_timeout_ms}
rpsreadserv.https.connectionTimeoutInMs=${default_connection_timeout_ms}
rpsreadserv.https.readTimeoutInMs=${default_timeout_ms}
rpsreadserv.https.sslHandshakeTimeoutInMs=${default_sslHandshake_timeout_ms}
rpsreadserv.https.requestTimeoutInMs=${default_timeout_ms}
rpsreadserv.POST.http.requestTimeoutInMs=${default_timeout_ms}
rpsreadserv.GET.http.requestTimeoutInMs=${default_timeout_ms}

```

/Dev/config/application-override.properties

```

rpsreadserv.baseUrl=https://msmaster.qa.paypal.com:14751

```

/QA/config/application-override.properties

```

rpsreadserv.baseUrl=https://msmaster.qa.paypal.com:14751

```

Sample rpsreadserv integration from raptor service:

<https://github.paypal.com/Campaigns-R/dynsegmentationserv/blob/develop/dynsegmentationservService/src/main/resources/META-INF/configuration/Common/config/application.properties#L17>

• **FAQs**

- a. **I created an account on stage and do not see the attributes expected to be updated in real time in RPS. The user is hence not qualifying for expected segments. What should i do?**
 - i. TBD
- b. **I have set all the attributes/lists used in the Rules in a Dynamic Segment. The user still does not qualify for the segment. What should i do?**
 - i. TBD
- c. **How can i request for a new attribute in RPS?**
 - i. TBD
- d. **How are attributes updated into RPS?**
 - i. TBD
- e. **Have Issues understanding the operators available for building segments.**
 - i. Refer to the following Section to understand what each operator stands for : [Capabilities in Segment Evaluation#OperatorstoDatatypemapping](#)